Computação Móvel - LEI 2025-2026	Flutter
Projeto Época Normal	

Projeto Prático

1. Introdução

Propõe-se o desenvolvimento de um projeto prático que permita consolidar os conhecimentos adquiridos ao longo do semestre.

O projeto consiste no design e implementação de uma aplicação *mobile* com **tema livre**, usando Flutter.

2. Objetivos

Os principais objetivos do projeto são:

- Desenvolver uma aplicação mobile completa e funcional;
- Aplicar conceitos de UI/UX;
- Integrar serviços externos;
- Implementar mecanismos de autenticação;
- Utilizar boas práticas de desenvolvimento colaborativo.

3. Requisitos Mínimos

Todas as aplicações deverão obrigatoriamente incluir:

1. **Autenticação e registo de utilizadores**
   (ex: email/password, OAuth, etc.)
2. **Sistema de notificações**
   (ex: push notifications ou notificações locais relevantes para a app)
3. **Integração com uma API externa**
   (ex: APIs públicas – meteorologia, mapas, filmes, desporto, etc.)
   Exemplos:
   https://github.com/public-apis/public-apis
   https://free-apis.github.io/#/
4. **Base de dados remota (não local)**
   (ex: Firebase Firestore, Realtime Database, ou equivalente)

4. Organização do Projeto

- O projeto deverá ser desenvolvido em **equipas de 4 elementos**
- Cada equipa deverá criar e manter um **repositório Git** (GitHub, GitLab ou equivalente), que será utilizado para:
    - Gestão de versões;
    - Colaboração entre elementos;
    - Avaliação do progresso do trabalho.

5. Entregas e Avaliação

5.1 Entrega Intermédia (20%) (4 de maio)

Objetivo: Avaliar o progresso inicial do projeto.

Deverá incluir:

- Definição do conceito da aplicação;
    - Descrição geral da app e público-alvo;
    - Descrição das funcionalidades gerais (aquelas funcionalidades que qualquer tipo de aplicação móvel possui, por exemplo, registo, autenticação, *splashscreen*, mostrar créditos dos autores, etc.);
    - Descrição das funcionalidades essenciais específicas do tema que respondam aos objetivos principais da app;
    - Indicação dos aspetos originais e das mais-valias;
- *Wireframes* / *mockups* / protótipos (ex: Figma);
- Esquema de navegação da app (pode ser feito diretamente no Figma);
- Implementação parcial do **layout (UI)**;
- Configuração inicial do projeto (incluindo repositório Git).

5.2 Entrega Final (30%) (9 de junho)

Objetivo: Avaliação do produto final.

Deverá incluir:

- Aplicação funcional completa;
- Implementação de todos os requisitos mínimos;
- Código organizado e documentado;
- Integração com API externa e base de dados remota;
- Demonstração da aplicação;
- Repositório Git atualizado e devidamente estruturado.

6. Critérios de Avaliação e Ponderação

Funcionalidade e Requisitos (30%)

- Correção do funcionamento de todas as funcionalidades principais.
- Presença de funcionalidades extras, se aplicável.
- Tratamento adequado de situações de erro ou excepções.

Arquitectura e estrutura do código (10%)

- Organização e modularização do código (pastas, arquivos, componentes).
- Uso adequado de padrões de projeto (ex: BLoC, Provider, MVC, Clean Architecture).
- Separação de responsabilidades (separação de UI e lógica de negócios).
- Boas práticas de codificação (nomenclatura, comentários, etc.).

Interface Gráfica (UI/UX) (10%)

- Estilização atraente e coesa (cores, fontes, espaçamento).
- Uso adequado de widgets do Flutter (StatefulWidget, StatelessWidget, etc.).
- Responsividade e adaptação a diferentes tamanhos de tela.
- Experiência do usuário (navegação intuitiva, feedbacks visuais, etc.).

Qualidade do Código (10%)

- Ausência de bugs críticos ou comportamentos inesperados.
- Uso de ferramentas de análise de código (como `dart analyze`).

Desempenho (5%)

- Velocidade de resposta da aplicação.
- Uso eficiente de recursos (memória, CPU).
- Ausência de *lags* ou travamentos.
- Uso de técnicas de otimização (como `const`, `ListView.builder`, etc.).

Integração com APIs ou Bases de dados - 2ª fase (10%)

- Correto uso de chamadas à API
- Uso de bibliotecas adequadas (como `http`, `dio`, `json_serializable`).
- Tratamento de respostas e erros de rede.
- Uso de persistência local (como `shared_preferences`, `sqflite`, `hive`, etc.)
- Envio e recepção adequada de notificações via Firebase

Documentação e entrega (15%)

- Documentação do código (comentários, explicações de funcionalidades).
- README do projeto (instalação, execução, funcionalidades).
- Explicação clara do funcionamento do projeto.
- Utilização adequada do *Git* (histórico de *commits*, organização, etc.);

Inovação e Originalidade (10%)

- Criatividade na abordagem e resolução de problemas.
- Soluções não triviais ou personalizadas.
- Inclusão de funcionalidades adicionais não solicitadas, mas úteis.

7. Considerações Finais

- Projetos incompletos ou que não cumpram os requisitos mínimos terão penalização;
- É valorizada a criatividade, mas também a robustez e qualidade técnica da solução;
- É permitido o uso de ferramentas de Inteligência Artificial (AI) e Large Language Models (LLMs) para apoio na geração de designs e código. No entanto, todos os elementos da equipa deverão demonstrar domínio dos conceitos e do código desenvolvido, sendo capazes de o explicar e defender durante a avaliação. A utilização destas ferramentas não substitui a compreensão do trabalho realizado. Situações em que se verifique falta de domínio do projeto poderão resultar em penalização na avaliação, incluindo a atribuição de classificação negativa.
- Poderá ser requerida uma prova de avaliação adicional, quando ocorram situações que levantem suspeitas de fraude.

Em situação de atraso na entrega, aplica-se a seguinte regra:

- 0,5 valores de desconto a aplicar no primeiro dia;
- 1 valor de desconto a aplicar no segundo dia, cumulado com os 0,5 valores do primeiro dia, totalizando 1,5 valores de desconto;
- **ao 3º dia de atraso e em diante não são aceites entregas de projecto.**